

PROPUESTA PARA PROYECTO DE GRADO

TÍTULO

ParaLab: Plataforma educativa para el aprendizaje práctico del paralelismo y la concurrencia

OBJETIVO GENERAL

Desarrollar una plataforma educativa que permita aprender de forma práctica el paralelismo y la concurrencia mediante una progresión guiada que va desde la programación de memoria compartida con OpenMP en una máquina, hasta la programación de memoria distribuida con MPI sobre contenedores que simulan múltiples nodos, acompañada de un módulo educativo y de la medición integrada del rendimiento.

ESTUDIANTE(S)

Javier Felipe Aldana Jaramillo

Documento	Celular	Correo Javeriano
cc. 1110366275	322-235-4026	al_javier@javeriana.edu.co

Sofia Carolina Mantilla Vega

Documento	Celular	Correo Javeriano
cc. 1034277746	304-607-2763	sofiamantilla@javeriana.edu.co

Jorge Humberto Sierra Laiton

Documento	Celular	Correo Javeriano
cc. 1034659161	302-854-0107	sierral-jorgeh@javeriana.edu.co

Sara Valentina García Granados

Documento	Celular	Correo Javeriano
cc. 1032936961	304-3669925	sv_garciag@javeriana.edu.co

DIRECTOR

Ing. Alejandro Castro Martinez

Documento	Celular	Correo Javeriano	Empresa donde trabaja y cargo
cc. 1019124610	310-249-8720	alejandrocastro@javeriana.edu.co;	Pontificia Universidad Javeriana; Profesor Departamento de Sistemas

Contenido

1	VISIÓN GLOBAL	5
1.1	ANTECEDENTES, PROBLEMA Y SOLUCIÓN PROPUESTA.....	5
1.1.1	<i>Descripción de la problemática u oportunidad</i>	5
1.1.2	<i>Formulación del problema.....</i>	6
1.1.3	<i>Propuesta de solución.....</i>	6
1.1.4	<i>Justificación de la solución.....</i>	7
1.2	DESCRIPCIÓN GENERAL DEL PROYECTO	8
1.2.1	<i>Objetivo general.....</i>	8
1.2.2	<i>Objetivos Específicos</i>	8
1.3	ENTREGABLES, ESTÁNDARES UTILIZADOS Y JUSTIFICACIÓN.....	9
2	ANÁLISIS DE IMPACTO.....	13
2.1.1	<i>Impacto esperado a corto plazo.....</i>	13
2.1.2	<i>Impacto esperado a mediano plazo.....</i>	13
2.1.3	<i>Impacto esperado a largo plazo.....</i>	14
2.1.4	<i>Impacto en la población u organización beneficiaria</i>	14
2.1.5	<i>Impacto social del proyecto</i>	15
3	PROCESO.....	16
3.1	FASE METODOLÓGICA 1: ANÁLISIS Y DISEÑO	17
3.1.1	<i>Método.....</i>	17
3.1.2	<i>Actividades</i>	17
3.1.3	<i>Resultados esperados.....</i>	17
3.2	FASE METODOLÓGICA 2: IMPLEMENTACIÓN	18
3.2.1	<i>Método.....</i>	18
3.2.2	<i>Actividades</i>	18
3.2.3	<i>Resultados esperados.....</i>	18
3.3	FASE METODOLÓGICA 3: EVALUACIÓN Y VALIDACIÓN	19
3.3.1	<i>Método.....</i>	19
3.3.2	<i>Actividades</i>	19
3.3.3	<i>Resultados esperados.....</i>	19
4	ASPECTOS GENERALES DEL PROYECTO.....	20
4.1	COMPROMISO DE APOYO DE LA INSTITUCIÓN	20
4.2	DERECHOS PATRIMONIALES	20
5	MARCO TEÓRICO.....	22
5.1	FUNDAMENTOS Y CONCEPTOS RELEVANTES PARA EL PROYECTO	22
5.2	ANÁLISIS DE ALTERNATIVAS DE SOLUCIÓN	26
5.2.1	<i>Alternativas de solución e impacto</i>	26

	<i>5.2.2 Comparación de alternativas.....</i>	<i>28</i>
6	REFERENCIAS	30

1 Visión global

1.1 Antecedentes, problema y solución propuesta

1.1.1 Descripción de la problemática u oportunidad

El paralelismo y la concurrencia se han convertido en competencias transversales del desarrollo de software contemporáneo. Prácticamente toda la computación moderna —procesadores multinúcleo, aceleradores gráficos, servicios distribuidos, motores de inteligencia artificial y procesamiento masivo de datos— depende de la capacidad de dividir un problema en tareas que se ejecutan de forma simultánea y de coordinarlas correctamente. En consecuencia, la formación de ingenieros y científicos de la computación exige no solo comprender los principios teóricos del paralelismo, sino también escribir, ejecutar, medir y razonar empíricamente sobre el comportamiento de programas concurrentes (Barrios-Hernández et al., 2024; Ngo & Kilgannon, 2020).

No obstante, su enseñanza tiende a concentrarse en la fundamentación conceptual y en ejercicios introductorios de programación, dejando por fuera competencias prácticas esenciales para trabajar con sistemas reales. Se identifican tres carencias recurrentes.

En primer lugar, la medición e interpretación del rendimiento: los estudiantes de ingeniería de sistemas o ciencias de la computación que ya tienen bases de programación, rara vez aprenden a obtener y analizar métricas como la aceleración (speedup), la eficiencia, la escalabilidad fuerte y débil, o las implicaciones de la ley de Amdahl, que son precisamente los instrumentos que permiten determinar si una solución paralela es realmente mejor que la secuencial.

En segundo lugar, la ausencia de un entorno de experimentación práctico, progresivo e iterativo, donde el estudiante escriba código, lo ejecute, observe su comportamiento, lo modifique y vuelva a medir, avanzando de lo simple a lo complejo.

En tercer lugar, el desconocimiento de las herramientas utilizadas en escenarios reales de programación paralela y distribuida; en muchos contextos formativos, la práctica puede quedar limitada a ejercicios introductorios, sin exponer al estudiante a modelos y herramientas de uso real, como la programación de memoria compartida (OpenMP), la de memoria distribuida mediante paso de mensajes (MPI), el uso de contenedores o los planificadores de tareas (Hebert et al., 2024; Barrios-Hernández et al., 2024).

La consecuencia es una brecha entre el conocimiento conceptual y su aplicación práctica: el estudiante comprende qué es el paralelismo, pero no necesariamente sabe evaluarlo, optimizarlo ni argumentar empíricamente sobre el comportamiento de una aplicación concurrente, ni reconoce las tecnologías con las que tendrá que trabajar (Hebert et al., 2024; Ngo & Kilgannon, 2020).

De esta situación surge una oportunidad concreta: diseñar un entorno controlado, accesible y reproducible que permita a los estudiantes recorrer una progresión de aprendizaje —desde el paralelismo de memoria compartida en una máquina hasta la ejecución distribuida entre nodos simulados—, midiendo el rendimiento en cada paso y familiarizándose con tecnologías actuales. La contenedorización y la simulación de un clúster son el laboratorio que hace posible esa práctica, no el fin.

1.1.2 Formulación del problema

De las carencias descritas en la sección anterior, una de las más determinantes es la limitada posibilidad de evaluar empíricamente el rendimiento de una solución paralela. Sin espacios adecuados para ejecutar, medir e interpretar resultados, el estudiante no puede responder preguntas fundamentales: si una versión paralela es realmente más rápida que su equivalente secuencial, cuánto escala al aumentar los recursos, qué parte del programa limita la mejora y qué patrón de paralelismo resulta más adecuado para un problema determinado. Esto hace que el aprendizaje permanezca incompleto y dificulta la transición entre la teoría y la práctica.

En consecuencia, se identifica una brecha entre la necesidad de formar profesionales capaces de escribir, medir y razonar sobre programas paralelos y concurrentes con tecnologías actuales, y la disponibilidad real de entornos prácticos que permitan desarrollar esas competencias de forma progresiva. El problema que orienta este proyecto se formula así: ¿cómo proporcionar a estudiantes de ingeniería un entorno práctico, accesible y reproducible que les permita aprender el paralelismo y la concurrencia de manera incremental, midiendo e interpretando el rendimiento y reconociendo las tecnologías actuales?

1.1.3 Propuesta de solución

Se propone el diseño e implementación de una plataforma educativa interactiva cuyo propósito central es el aprendizaje práctico y teórico del paralelismo y la concurrencia, organizada como una progresión didáctica que recorre los dos paradigmas fundamentales de la programación paralela. La propuesta se enmarca en el área de ingeniería de software y sistemas distribuidos.

El primer paradigma es el de memoria compartida, en el que varios hilos de ejecución acceden a un mismo espacio de memoria y deben coordinarse mediante mecanismos de sincronización; se aborda en un primer nivel con OpenMP sobre una sola máquina, donde el estudiante observa de forma directa la creación de hilos, el reparto del trabajo y los efectos de la sincronización.

El segundo paradigma es el de memoria distribuida, en el que procesos independientes no comparten memoria y se coordinan exclusivamente mediante paso de mensajes; se aborda en un segundo nivel con MPI, distribuyendo la ejecución entre múltiples contenedores que simulan nodos independientes aunque la ejecución se realiza sobre una única máquina física, cada contenedor representa un nodo lógico independiente dentro del entorno simulado, lo que permite experimentar con la comunicación entre procesos sin disponer de varias máquinas físicas. La secuencia memoria compartida hacia una memoria distribuida no es arbitraria: introduce primero el modelo más intuitivo (hilos que comparten datos) y luego el más abstracto (procesos que se comunican por mensajes), siguiendo el orden natural de complejidad conceptual con que se introducen estos modelos en la formación en computación paralela.

En cada nivel, la plataforma permite obtener y visualizar métricas de rendimiento (aceleración, eficiencia y escalabilidad, entre otras) y contrastar configuraciones, de modo que la medición y la interpretación dejen de ser un contenido teórico para convertirse en una práctica directa sobre código propio. El componente técnico que habilita esta práctica es un simulador de clúster basado en contenedores, que reproduce de forma controlada un entorno de ejecución distribuida sin requerir infraestructura física especializada; la contenedorización permite ofrecer entornos reproducibles y portables (Chung et al., 2016; Kurtzer et al., 2017; Sheka et al., 2019). El simulador no es el objeto de aprendizaje, sino el laboratorio que lo hace posible.

La solución se complementa con material educativo organizado en dos componentes: un módulo de talleres prácticos guiados y un módulo de recursos en video, ambos orientados a cerrar las carencias identificadas: cómo se obtienen e interpretan las métricas, en qué se diferencian los paradigmas de memoria compartida y memoria distribuida, y cómo se usa la propia aplicación. Como extensión opcional, y solo si el alcance temporal lo permite, se contempla un tercer nivel que incorpore un planificador de tareas tipo SLURM para aproximar al estudiante a la gestión de trabajos en entornos reales; este nivel se plantea como trabajo futuro y no como compromiso del proyecto.

1.1.4 Justificación de la solución

La solución propuesta es adecuada porque ataca de forma directa la brecha identificada en las secciones anteriores: convierte en práctica medible y progresiva aquello que hoy se enseña de manera predominantemente teórica. Su pertinencia se sustenta en cuatro frentes complementarios. A diferencia de enfoques centrados únicamente en disponer un entorno de ejecución, esta propuesta integra la ejecución de código, la visualización de métricas y una ruta pedagógica guiada, de manera que el estudiante no solo corre programas paralelos, sino que aprende a interpretar su comportamiento.

Desde el punto de vista pedagógico, la propuesta se justifica porque ofrece un entorno donde conceptos que normalmente solo se abordan de forma teórica —comunicación entre procesos, distribución de tareas, sincronización, aceleración, eficiencia y escalabilidad— pueden practicarse y medirse de manera concreta y en una secuencia que va de lo simple a lo complejo. Diversos trabajos muestran que los entornos virtuales y los clústeres experimentales acercan a los estudiantes a la programación paralela de forma efectiva (Ngo & Kilgannon, 2020; Demay et al., 2024); esta propuesta extiende esa idea al poner el énfasis no solo en ejecutar, sino en medir, interpretar y avanzar progresivamente, que es la competencia práctica hoy ausente.

Desde el punto de vista técnico, la contenedorización proporciona reproducibilidad y portabilidad, condiciones necesarias para que las mediciones sean comparables y para que el entorno se despliegue de manera consistente en distintos equipos (Chung et al., 2016; Kurtzer et al., 2017; Zhou et al., 2023). El uso de un simulador de clúster como medio permite, además, exponer al estudiante a tecnologías representativas —modelos de memoria compartida y distribuida y, opcionalmente, planificación de tareas— sin la complejidad ni el costo de una infraestructura física dedicada (Greeneche & Cerin, 2022).

Desde una perspectiva profesional y social, las competencias en paralelismo y concurrencia tienen una demanda creciente en áreas como la ciencia de datos, la inteligencia artificial y la simulación computacional, donde no basta con escribir código paralelo, sino que se requiere saber evaluarlo y optimizarlo (Barrios-Hernández et al., 2024; Hebert et al., 2024). Finalmente, la propuesta se dirige a una población concreta con una necesidad identificable: estudiantes y docentes de programas afines a ingeniería y ciencias de la computación que requieren un espacio accesible para aprender paralelismo de forma progresiva, medir su rendimiento y conocer tecnologías actuales. Al ofrecer una plataforma controlada, reproducible y de baja barrera de entrada, la solución articula valor educativo, pertinencia técnica y proyección profesional en torno a un eje claro: el aprendizaje práctico del paralelismo y la concurrencia, para el cual el entorno basado en contenedores actúa como medio y no como fin.

1.2 Descripción general del proyecto

El proyecto comprende el desarrollo de una plataforma educativa interactiva orientada al aprendizaje práctico del paralelismo y la concurrencia, concebida como un entorno controlado donde el estudiante avanza de manera progresiva desde los conceptos más intuitivos hasta los más abstractos. La plataforma estará compuesta por un entorno de ejecución local para OpenMP, un entorno distribuido basado en contenedores para MPI, una interfaz gráfica de configuración y ejecución, un módulo de visualización de métricas y un módulo educativo con talleres y recursos audiovisuales. La plataforma no se limita a ejecutar código: integra la medición del rendimiento y un módulo educativo, de modo que el aprendizaje articule práctica, evaluación y fundamentación.

La plataforma se organiza en una progresión didáctica de dos niveles. En el primer nivel, el estudiante trabaja con el paralelismo de memoria compartida mediante OpenMP en una sola máquina, donde puede observar de forma directa el comportamiento de la ejecución concurrente, la creación de hilos y la sincronización. En el segundo nivel, escala al paralelismo de memoria distribuida con MPI, donde la ejecución se reparte entre múltiples contenedores que simulan nodos independientes, permitiendo experimentar con la comunicación entre procesos sin requerir varias máquinas físicas.

Desde el punto de vista técnico, el componente que habilita esta práctica es un simulador de clúster basado en contenedores. Sobre estos contenedores se disponen las dependencias, bibliotecas y componentes necesarios para soportar la ejecución de aplicaciones paralelas y la comunicación entre nodos simulados. El uso de contenedores permite mantener configuraciones de software uniformes, aislar los componentes del sistema y desplegar el entorno de manera consistente en distintos equipos, lo que favorece la reproducibilidad de las mediciones y la portabilidad de la plataforma.

En términos de funcionamiento, la plataforma permite cargar y ejecutar aplicaciones paralelas, configurar el entorno mediante una interfaz gráfica sin editar archivos manualmente, y obtener métricas de rendimiento —aceleración, eficiencia y escalabilidad— que el estudiante puede visualizar e interpretar. En conjunto, el proyecto se configura como una plataforma formativa y de experimentación que articula ejecución, medición y fundamentación en torno a un eje claro: el aprendizaje práctico del paralelismo y la concurrencia, donde el entorno basado en contenedores actúa como medio y no como fin.

1.2.1 Objetivo general

Desarrollar una plataforma educativa interactiva que facilite el aprendizaje práctico de conceptos de paralelismo y concurrencia, integrando programación de memoria compartida con OpenMP y programación de memoria distribuida con MPI, mediante el uso de contenedores que simulen múltiples nodos de cómputo.

1.2.2 Objetivos Específicos

Los siguientes objetivos específicos cumplen con los criterios SMART y constituyen metas parciales que apoyan el cumplimiento del objetivo general:

1. Analizar los conceptos, herramientas y requerimientos necesarios para el aprendizaje práctico del paralelismo y la concurrencia, considerando el uso de OpenMP, MPI y contenedores como base tecnológica de la plataforma.
2. Diseñar una plataforma educativa interactiva que integre contenidos formativos, ejercicios prácticos y entornos de ejecución orientados a la programación paralela y distribuida.
3. Implementar los módulos prácticos de la plataforma y el entorno basado en contenedores, permitiendo la ejecución de programas con OpenMP y MPI sobre nodos de cómputo simulados.
4. Evaluar el funcionamiento y la utilidad de la plataforma mediante pruebas técnicas y actividades de validación con usuarios o casos de prueba.

Nota: la integración de un planificador de tareas tipo SLURM se contempla como extensión opcional y trabajo futuro (un eventual tercer nivel), no como objetivo comprometido, dado su carácter incierto dentro del alcance temporal del proyecto.

1.3 Entregables, estándares utilizados y justificación

Entregable	Estándares asociados	Justificación
Aplicación ParaLab Plataforma educativa interactiva para el aprendizaje práctico del paralelismo y la concurrencia	OCI (Open Container Initiative) OpenMP API Specification MPI Standard IEEE 1484.12.1 (LOM)	El entregable principal es la plataforma ParaLab, orientada a que los estudiantes aprendan conceptos de paralelismo y concurrencia mediante la ejecución práctica de programas. La plataforma integra un entorno local para programas con OpenMP, un entorno basado en contenedores para programas MPI, una interfaz gráfica de configuración y ejecución, un módulo de visualización de métricas y recursos educativos. OCI respalda el uso de contenedores portables y reproducibles; OpenMP y MPI orientan los modelos de programación paralela trabajados; IEEE 1484.12.1 permite estructurar los contenidos educativos como objetos de aprendizaje reutilizables.
Entorno de ejecución para memoria compartida con OpenMP	OpenMP API Specification ISO/IEC/IEEE 29119	Este entregable corresponde al entorno que permitirá ejecutar programas de memoria compartida en una máquina, usando OpenMP como tecnología principal. Su propósito es que el estudiante observe la creación de hilos, el reparto de tareas, la sincronización y el comportamiento de una solución paralela frente a una versión secuencial. OpenMP funciona como referencia técnica para la programación de memoria compartida, mientras que ISO/IEC/IEEE 29119 orienta la documentación de pruebas funcionales para verificar que los programas se ejecuten correctamente.
Entorno de ejecución distribuida con MPI sobre contenedores	OCI (Open Container Initiative) MPI Standard IEEE 802	Este entregable corresponde al entorno distribuido en el que varios contenedores simulan nodos de cómputo independientes para ejecutar programas MPI. Su propósito es permitir que el estudiante experimente con memoria

		distribuida, comunicación entre procesos y ejecución sobre nodos simulados sin requerir infraestructura física especializada. OCI respalda la portabilidad y consistencia de los contenedores; MPI define el modelo de paso de mensajes utilizado; IEEE 802 se relaciona con la comunicación de red entre los contenedores dentro del entorno simulado.
Módulo de visualización de métricas de rendimiento	ISO/IEC 25023	Este entregable permite presentar métricas como aceleración, eficiencia, tiempos de ejecución y escalabilidad, con el fin de que el estudiante no solo ejecute programas paralelos, sino que también pueda interpretar su comportamiento. ISO/IEC 25023 sirve como referencia para estructurar mediciones cuantitativas de calidad y rendimiento del sistema, favoreciendo que los indicadores presentados sean consistentes, comprensibles y comparables entre diferentes configuraciones.
Módulo educativo de talleres prácticos y recursos audiovisuales	IEEE 1484.12.1 (LOM) IEEE 26511	Este entregable incluye talleres guiados, ejercicios paso a paso, videos explicativos y materiales de apoyo relacionados con memoria compartida, memoria distribuida, sincronización, comunicación entre procesos, métricas de rendimiento y uso de la plataforma. IEEE 1484.12.1 permite organizar los contenidos como recursos educativos digitales, facilitando su clasificación y reutilización. IEEE 26511 orienta la elaboración de documentación clara para usuarios finales.
Documentación de arquitectura y diseño de software	UML 2.5 BPMN 2.0 IEEE 1016	Este entregable documenta la arquitectura de ParaLab, incluyendo sus componentes principales, relaciones, flujos de ejecución e interacción del usuario con la plataforma. UML 2.5 se utiliza para representar diagramas de componentes, despliegue, secuencia o

		paquetes; BPMN 2.0 permite modelar los flujos de proceso asociados a la ejecución de programas y uso de la plataforma; IEEE 1016 orienta la descripción formal de decisiones de diseño de software.
Documentación de pruebas funcionales y validación de la plataforma	ISO/IEC/IEEE 29119 ISO 9241-11	Este entregable reúne los casos de prueba, resultados de pruebas funcionales y actividades de validación con usuarios o casos de prueba. ISO/IEC/IEEE 29119 permite estructurar las pruebas del software, criterios de aceptación y evidencias de funcionamiento. ISO 9241-11 permite evaluar la usabilidad de la plataforma considerando eficacia, eficiencia y satisfacción del usuario.
Manual de usuario y documentación técnica de instalación y uso	IEEE 26511	Este entregable contiene las instrucciones necesarias para instalar, configurar y utilizar ParaLab, incluyendo el uso de la interfaz gráfica, la ejecución de programas OpenMP y MPI, la interpretación de métricas y el acceso a los recursos educativos. IEEE 26511 orienta la creación de documentación de usuario clara, completa y comprensible para estudiantes, docentes y usuarios finales.
Resultados de validación educativa	ISO 9241-11 IEEE 1484.12.1 (LOM)	Este entregable presenta los resultados obtenidos al evaluar la utilidad de la plataforma en el contexto educativo. Puede incluir retroalimentación de estudiantes o docentes, observaciones de uso, resultados de actividades prácticas y evidencias sobre la comprensión de los conceptos trabajados. ISO 9241-11 permite valorar la experiencia de uso, mientras que IEEE 1484.12.1 respalda la organización y evaluación de los recursos educativos integrados.

Tabla 1: Entregables del proyecto. Elaboración propia.

2 Análisis de impacto

En caso de que el proyecto sea exitoso, se espera que ParaLab genere un impacto progresivo en la enseñanza y el aprendizaje práctico del paralelismo y la concurrencia, inicialmente dentro de la Pontificia Universidad Javeriana y, posteriormente, en otros contextos académicos donde se requieran herramientas accesibles para experimentar con programación paralela y distribuida.

La plataforma busca fortalecer la comprensión de conceptos como memoria compartida, memoria distribuida, sincronización, comunicación entre procesos, aceleración, eficiencia y escalabilidad, mediante un entorno práctico que integra ejecución de código, visualización de métricas y recursos educativos. A continuación, se describe el impacto esperado en diferentes horizontes temporales, así como su efecto sobre la población beneficiaria y su dimensión social.

2.1.1 Impacto esperado a corto plazo

En el corto plazo, el principal impacto se concentra en la Pontificia Universidad Javeriana, donde se origina el proyecto. ParaLab podrá ser utilizado como una herramienta de apoyo para estudiantes de Ingeniería de Sistemas y programas afines que necesiten acercarse de manera práctica a los conceptos de paralelismo y concurrencia.

La plataforma permitirá que los estudiantes experimenten con programación de memoria compartida mediante OpenMP en una máquina local y con programación de memoria distribuida mediante MPI sobre contenedores que simulan nodos de cómputo. Esto facilitará que conceptos que usualmente se explican de manera teórica puedan observarse directamente mediante la ejecución de programas, la comparación de resultados y la interpretación de métricas de rendimiento.

Para los docentes, ParaLab podrá servir como recurso didáctico para acompañar clases relacionadas con programación paralela, sistemas distribuidos, arquitectura de computadores o computación de alto rendimiento introductoria. Al integrar ejercicios, recursos educativos y visualización de métricas, la plataforma facilitará la preparación de actividades prácticas y permitirá conectar la teoría con la experimentación en un entorno controlado y accesible.

2.1.2 Impacto esperado a mediano plazo

A mediano plazo, ParaLab podrá contribuir a fortalecer la formación práctica en paralelismo y concurrencia dentro de diferentes asignaturas del programa de Ingeniería de Sistemas y áreas relacionadas. Su uso puede permitir que los estudiantes desarrollen habilidades no solo para escribir programas paralelos, sino también para medir su comportamiento, interpretar resultados y comprender las diferencias entre los modelos de memoria compartida y memoria distribuida.

Además, al estar basada en contenedores, la plataforma puede facilitar su despliegue en distintos equipos sin requerir infraestructura física especializada. Esto puede favorecer su uso en laboratorios académicos, espacios de aprendizaje autónomo o actividades guiadas por docentes, reduciendo las barreras técnicas que normalmente dificultan la experimentación con programación paralela y distribuida.

En este horizonte, también se espera que la plataforma pueda ser ajustada o ampliada a partir de la retroalimentación de usuarios, incorporando nuevos ejercicios, ejemplos, recursos educativos o mejoras en la visualización de métricas. De esta manera, ParaLab podría consolidarse como una herramienta flexible para apoyar procesos de enseñanza práctica en temas de concurrencia, paralelismo y ejecución distribuida.

2.1.3 Impacto esperado a largo plazo

A largo plazo, ParaLab podría convertirse en un recurso educativo reutilizable para la enseñanza de conceptos fundamentales de paralelismo y concurrencia en contextos universitarios. Su enfoque progresivo, que inicia con memoria compartida mediante OpenMP y avanza hacia memoria distribuida con MPI sobre contenedores, puede servir como base para introducir a los estudiantes en modelos de programación usados en escenarios reales de computación paralela y distribuida.

Aunque la plataforma no busca reemplazar infraestructuras especializadas ni clústeres físicos de alto rendimiento, sí puede preparar mejor a los estudiantes para comprender su funcionamiento antes de interactuar con entornos más complejos. En ese sentido, ParaLab puede funcionar como un puente entre la teoría vista en clase y la práctica requerida para trabajar posteriormente con sistemas distribuidos, aplicaciones paralelas o entornos de computación de mayor escala.

Con el tiempo, la plataforma podría ser extendida con nuevos módulos, ejercicios o niveles de complejidad, incluyendo eventualmente herramientas de planificación de tareas como SLURM, siempre que el alcance futuro del proyecto lo permita. Esto abriría la posibilidad de que ParaLab evolucione como una herramienta educativa más completa para la formación en programación paralela, computación distribuida y fundamentos de computación de alto rendimiento.

2.1.4 Impacto en la población u organización beneficiaria

La población directamente beneficiada está compuesta por estudiantes y docentes de programas afines a Ingeniería de Sistemas, Ciencias de la Computación y áreas relacionadas. Para los estudiantes, ParaLab ofrece un espacio accesible donde pueden practicar conceptos de paralelismo y concurrencia mediante la ejecución de programas, la configuración de escenarios de prueba y la observación de métricas de rendimiento.

El beneficio principal para los estudiantes consiste en pasar de una comprensión únicamente conceptual a una comprensión práctica y medible. Al interactuar con programas OpenMP y MPI, los estudiantes pueden observar cómo se comportan los hilos, procesos, nodos simulados y mecanismos de comunicación, así como analizar si una solución paralela realmente mejora el desempeño frente a una versión secuencial.

Para los docentes, la plataforma representa una herramienta pedagógica que permite diseñar actividades prácticas, explicar conceptos complejos con apoyo visual y evaluar la comprensión de los estudiantes en un entorno controlado. Además, al integrar recursos educativos y ejercicios guiados, ParaLab puede reducir el tiempo de preparación técnica necesario para realizar prácticas de programación paralela y distribuida.

En el contexto de la Pontificia Universidad Javeriana, el proyecto puede aportar al fortalecimiento de la formación práctica en áreas relacionadas con sistemas distribuidos, arquitectura de computadores, programación concurrente y computación paralela. Esto contribuye a que los estudiantes desarrollen competencias más cercanas a las necesidades actuales del desarrollo de software, la ciencia de datos, la inteligencia artificial y la simulación computacional.

2.1.5 Impacto social del proyecto

Desde una perspectiva social, ParaLab contribuye a democratizar el acceso al aprendizaje práctico de conceptos de paralelismo y concurrencia, los cuales suelen requerir entornos técnicos especializados o configuraciones complejas para ser experimentados de manera adecuada. Al ofrecer una plataforma basada en contenedores y orientada al aprendizaje guiado, el proyecto reduce barreras de entrada para estudiantes que no cuentan con acceso directo a infraestructuras avanzadas.

El proyecto también favorece una formación más equitativa, ya que permite que más estudiantes puedan experimentar con programación paralela y distribuida desde un entorno controlado y reproducible. Esto resulta importante en contextos educativos donde el acceso a recursos especializados puede ser limitado o depender de disponibilidad institucional.

Adicionalmente, al fortalecer competencias en paralelismo, concurrencia, medición de rendimiento y ejecución distribuida, ParaLab puede contribuir indirectamente a la formación de profesionales mejor preparados para enfrentar problemas tecnológicos actuales. Estas competencias son relevantes en áreas como inteligencia artificial, procesamiento de datos, simulación científica, sistemas distribuidos y desarrollo de software de alto desempeño.

En conjunto, el impacto social del proyecto se relaciona con ampliar las oportunidades de aprendizaje práctico, reducir la brecha entre teoría y aplicación, y apoyar la formación de estudiantes con habilidades técnicas más sólidas para participar en contextos académicos, científicos y profesionales donde la computación paralela y distribuida tiene una importancia creciente.

3 Proceso

Para el desarrollo del proyecto se propone un enfoque metodológico híbrido que combina análisis y diseño de sistemas, desarrollo iterativo e incremental, y validación continua con usuarios o casos de prueba. Este enfoque permite abordar de manera ordenada la construcción técnica de la plataforma ParaLab y, al mismo tiempo, verificar su utilidad como herramienta educativa para el aprendizaje práctico del paralelismo y la concurrencia.

El componente de análisis y diseño permite identificar los conceptos, herramientas y requerimientos necesarios para construir una plataforma orientada al aprendizaje de memoria compartida con OpenMP, memoria distribuida con MPI y uso de contenedores para simular nodos de cómputo. A partir de este análisis se define la arquitectura general de la plataforma, los módulos principales, los flujos de interacción y los criterios de funcionamiento.

El componente iterativo e incremental permite desarrollar la solución por etapas, implementando primero los módulos esenciales y luego integrando progresivamente nuevas funcionalidades. Esto resulta adecuado para el proyecto porque ParaLab está compuesta por varios elementos relacionados entre sí: entorno de ejecución para OpenMP, entorno de ejecución distribuida con MPI sobre contenedores, interfaz gráfica, visualización de métricas y módulo educativo.

De manera complementaria, el componente de validación continua permite evaluar tanto el funcionamiento técnico de la plataforma como su utilidad en el contexto educativo. A través de pruebas técnicas, casos de prueba y actividades con usuarios, se busca verificar que la plataforma permita ejecutar los programas definidos, presentar métricas comprensibles y apoyar el aprendizaje práctico de los conceptos trabajados.

Este enfoque metodológico permite que el proyecto avance de forma progresiva, reduciendo riesgos técnicos y asegurando que cada fase contribuya directamente al cumplimiento de los objetivos específicos.

Relación entre metodologías

El análisis y diseño de sistemas permite establecer las bases conceptuales, técnicas y funcionales de ParaLab antes de su implementación. Esta etapa asegura que la plataforma responda a la problemática identificada y que sus componentes estén alineados con el objetivo de facilitar el aprendizaje práctico del paralelismo y la concurrencia.

El desarrollo iterativo e incremental permite construir la plataforma por módulos, evitando depender de una implementación completa desde el inicio. Cada iteración puede producir un avance funcional, como la ejecución de programas OpenMP, la configuración de contenedores para MPI, la visualización de métricas o la integración de contenidos educativos.

La validación continua permite revisar los avances del sistema desde dos perspectivas: la técnica y la educativa. La perspectiva técnica verifica que los módulos funcionen correctamente, mientras que la perspectiva educativa analiza si la plataforma resulta comprensible, útil y adecuada para estudiantes o docentes.

Estos tres enfoques se integran en el proyecto porque cada fase combina actividades de análisis, construcción y revisión. De esta manera, ParaLab puede evolucionar de forma controlada,

manteniendo coherencia entre los requerimientos, la implementación y la validación de su utilidad.

3.1 Fase metodológica 1: Análisis y diseño

Esta fase tiene como objetivo establecer las bases conceptuales, técnicas y estructurales del proyecto. En ella se analizan los conceptos de paralelismo y concurrencia que serán abordados por la plataforma, se identifican las herramientas tecnológicas necesarias y se define el diseño general de ParaLab.

3.1.1 Método

Se emplea un enfoque de análisis y diseño de sistemas, orientado a transformar las necesidades educativas y técnicas del proyecto en una solución estructurada. Este método permite definir los requerimientos de la plataforma, sus componentes principales, la arquitectura del sistema y los flujos de interacción del usuario.

3.1.2 Actividades

Las actividades de esta fase se orientan a comprender el problema y diseñar la solución. En primer lugar, se analizan los conceptos fundamentales que serán trabajados en la plataforma, como memoria compartida, memoria distribuida, sincronización, comunicación entre procesos, aceleración, eficiencia y escalabilidad.

Posteriormente, se estudian las herramientas tecnológicas que soportarán la solución, especialmente OpenMP para programación de memoria compartida, MPI para programación de memoria distribuida y contenedores para simular nodos de cómputo. También se identifican los requerimientos funcionales y no funcionales de la plataforma, considerando aspectos como ejecución de programas, configuración del entorno, visualización de métricas, facilidad de uso y portabilidad.

Finalmente, se diseña la arquitectura general de ParaLab, definiendo los módulos principales de la plataforma: entorno de ejecución local con OpenMP, entorno distribuido con MPI sobre contenedores, interfaz gráfica de configuración y ejecución, módulo de visualización de métricas y módulo educativo con talleres y recursos de apoyo.

3.1.3 Resultados esperados

Como resultado de esta fase se espera obtener la caracterización de los conceptos y herramientas necesarias para el proyecto, la definición de requerimientos educativos y técnicos, y el diseño inicial de la arquitectura de ParaLab. Estos resultados sirven como base para la implementación de los módulos prácticos y del entorno de ejecución de la plataforma.

3.2 Fase metodológica 2: Implementación

Esta fase tiene como objetivo construir los módulos principales de ParaLab de acuerdo con el diseño definido en la fase anterior. El desarrollo se realiza de manera iterativa e incremental, permitiendo implementar, probar e integrar progresivamente los componentes de la plataforma.

3.2.1 Método

Se adopta un enfoque iterativo e incremental de desarrollo de software. Este método permite dividir la implementación en ciclos de trabajo, en los cuales se construyen funcionalidades específicas, se prueban individualmente y luego se integran al sistema general. De esta forma, se reducen riesgos técnicos y se facilita la mejora continua de la plataforma.

3.2.2 Actividades

Las actividades de esta fase se organizan alrededor de los módulos principales de ParaLab. En primer lugar, se implementa el entorno de ejecución para programas de memoria compartida con OpenMP, permitiendo correr ejemplos y ejercicios en una máquina local.

En segundo lugar, se implementa el entorno de ejecución distribuida con MPI sobre contenedores, de manera que varios contenedores representen nodos de cómputo simulados y permitan experimentar con comunicación entre procesos. Esta actividad incluye la configuración de dependencias, bibliotecas y mecanismos de comunicación necesarios para ejecutar programas MPI en el entorno definido.

De manera paralela, se desarrolla la interfaz gráfica de la plataforma, orientada a facilitar la configuración del entorno, la ejecución de programas y la interacción del usuario sin depender exclusivamente de comandos o edición manual de archivos. También se implementa el módulo de visualización de métricas, encargado de presentar información como tiempos de ejecución, aceleración, eficiencia y escalabilidad.

Finalmente, se integra el módulo educativo, compuesto por talleres prácticos, ejercicios guiados y recursos audiovisuales que apoyan la comprensión de los conceptos abordados y el uso de la plataforma.

3.2.3 Resultados esperados

Como resultado de esta fase se espera obtener una versión funcional de ParaLab que integre los módulos principales: ejecución de programas OpenMP, ejecución de programas MPI sobre contenedores, interfaz gráfica de configuración y ejecución, visualización de métricas y recursos educativos. Estos resultados se relacionan directamente con el objetivo de implementar los módulos prácticos y el entorno basado en contenedores.

3.3 Fase metodológica 3: Evaluación y validación

Esta fase tiene como objetivo evaluar el funcionamiento técnico y la utilidad educativa de ParaLab. Para ello, se realizan pruebas funcionales, casos de prueba y actividades de validación que permitan verificar si la plataforma cumple con los objetivos propuestos.

3.3.1 Método

Se emplea un enfoque de evaluación basado en pruebas técnicas y validación con usuarios o casos de prueba. Las pruebas técnicas permiten verificar que los módulos de la plataforma funcionen correctamente, mientras que la validación educativa permite analizar si la plataforma resulta útil, comprensible y pertinente para apoyar el aprendizaje práctico del paralelismo y la concurrencia.

3.3.2 Actividades

Las actividades de esta fase incluyen la ejecución de pruebas funcionales sobre los módulos principales de ParaLab. Se verifica que los programas OpenMP puedan ejecutarse correctamente en el entorno local, que los programas MPI puedan ejecutarse sobre contenedores que simulan nodos de cómputo y que la interfaz gráfica permita configurar y ejecutar los escenarios definidos.

3.3.3 Resultados esperados

Como resultado de esta fase se espera contar con evidencia documentada del funcionamiento técnico de ParaLab, resultados de pruebas funcionales, retroalimentación de usuarios o casos de prueba y una valoración de la utilidad educativa de la plataforma. Estos resultados permiten verificar el cumplimiento del objetivo específico relacionado con la evaluación del funcionamiento y la utilidad de la solución.

4 Aspectos generales del proyecto

4.1 Compromiso de apoyo de la Institución

Para el desarrollo y validación del proyecto ParaLab, la Pontificia Universidad Javeriana brindará apoyo académico e institucional a través de los recursos disponibles en el Departamento de Ingeniería de Sistemas. Este apoyo permitirá orientar el desarrollo de la plataforma, validar su pertinencia educativa y realizar pruebas en entornos representativos de uso académico.

En primer lugar, se contará con el acompañamiento del director del proyecto y de profesores del área de sistemas distribuidos, arquitectura de computadores, programación paralela o áreas afines. Este acompañamiento será fundamental para revisar la coherencia técnica y pedagógica de la plataforma, así como para orientar la selección de conceptos, ejercicios y recursos educativos relacionados con paralelismo, concurrencia, OpenMP, MPI y contenedores.

En segundo lugar, se contempla el uso de equipos de cómputo de la universidad o de los laboratorios disponibles para realizar pruebas de instalación, ejecución y compatibilidad de la plataforma. Estas pruebas permitirán verificar que ParaLab pueda ejecutarse en equipos con diferentes configuraciones de hardware y software, favoreciendo su portabilidad y facilidad de despliegue en contextos académicos.

Adicionalmente, cuando sea pertinente, se podrá tomar como referencia la infraestructura institucional relacionada con computación de alto rendimiento, como el clúster ZINE, con el fin de contrastar conceptualmente algunos elementos de la plataforma con entornos reales de ejecución paralela o distribuida. Sin embargo, el proyecto no depende directamente del uso permanente de dicha infraestructura, ya que ParaLab está concebido como una plataforma educativa basada en ejecución local, OpenMP, MPI y contenedores que simulan nodos de cómputo.

Finalmente, se espera contar con la participación de estudiantes o docentes en actividades de validación, pruebas de uso o revisión de contenidos educativos. Esta retroalimentación permitirá evaluar la claridad, utilidad y pertinencia de la plataforma como herramienta de apoyo para el aprendizaje práctico del paralelismo y la concurrencia.

4.2 Derechos patrimoniales

Los derechos morales sobre el proyecto ParaLab corresponden a los estudiantes autores del trabajo de grado: Javier Felipe Aldana Jaramillo, Sofia Carolina Mantilla Vega, Jorge Humberto Sierra Laiton y Sara Valentina García Granados.

En lo referente a los derechos patrimoniales, dado que ParaLab es un producto de software desarrollado como trabajo de grado dentro de la Pontificia Universidad Javeriana y orientado a fines educativos, los derechos patrimoniales se sujetarán a las disposiciones institucionales aplicables para trabajos de grado y productos académicos desarrollados en el marco de la universidad.

El software y la documentación asociada podrán ser utilizados con fines académicos, de enseñanza, validación e investigación, de acuerdo con los lineamientos definidos por la Pontificia Universidad Javeriana. Los estudiantes conservarán el derecho a ser reconocidos como autores del proyecto en cualquier uso, modificación, presentación o distribución académica del producto.

5 Marco teórico

En esta sección se presentan los fundamentos conceptuales y técnicos necesarios para comprender el problema abordado en el proyecto y la solución propuesta. Dado que ParaLab se orienta al aprendizaje práctico del paralelismo y la concurrencia, el marco teórico se centra en los conceptos asociados a la programación paralela, los modelos de memoria compartida y memoria distribuida, las herramientas OpenMP y MPI, el uso de contenedores como medio de experimentación, la medición del rendimiento y el papel de los recursos educativos digitales en procesos de formación práctica.

El objetivo de esta sección es proporcionar una base conceptual que permita interpretar la propuesta desde dos perspectivas: en primer lugar, la perspectiva técnica, relacionada con los modelos y herramientas que permiten ejecutar programas paralelos y distribuidos; y, en segundo lugar, la perspectiva educativa, relacionada con la necesidad de que los estudiantes no solo comprendan estos conceptos de forma teórica, sino que puedan aplicarlos, medirlos e interpretarlos en un entorno controlado.

5.1 Fundamentos y conceptos relevantes para el proyecto.

Para comprender el alcance de ParaLab, es necesario definir los conceptos principales asociados al paralelismo, la concurrencia, la programación paralela y distribuida, el uso de contenedores y la medición del rendimiento. Estos fundamentos permiten que un lector no experto entienda tanto la problemática abordada como el valor de la solución propuesta.

- **Paralelismo:** se refiere a la ejecución simultánea de varias tareas o partes de una misma tarea con el propósito de reducir el tiempo total de procesamiento. En programación paralela, un problema se divide en unidades más pequeñas que pueden ejecutarse al mismo tiempo, aprovechando recursos como núcleos de procesamiento, hilos o nodos de cómputo.
- **Concurrencia:** se refiere a la capacidad de un sistema para gestionar múltiples tareas que progresan durante un mismo periodo de tiempo. A diferencia del paralelismo, la concurrencia no siempre implica ejecución simultánea real, sino coordinación entre tareas que pueden alternarse o ejecutarse en paralelo dependiendo de los recursos disponibles. Este concepto es fundamental para comprender problemas como sincronización, acceso compartido a datos, condiciones de carrera y comunicación entre procesos.
- **Programación paralela:** es el enfoque de programación orientado a dividir un problema en varias partes que pueden ejecutarse de forma simultánea. Su objetivo es aprovechar mejor los recursos computacionales disponibles y mejorar el rendimiento frente a una implementación secuencial. En el contexto de ParaLab, la programación paralela se aborda desde dos modelos principales: memoria compartida y memoria distribuida.
- **Memoria compartida:** es un modelo de programación paralela en el que varios hilos de ejecución acceden a un mismo espacio de memoria. Este modelo permite que los hilos compartan datos directamente, pero también exige mecanismos de sincronización para evitar errores cuando varias tareas intentan acceder o modificar la misma información al mismo tiempo.

- OpenMP: es una interfaz de programación utilizada para desarrollar aplicaciones paralelas en sistemas de memoria compartida. Permite crear y administrar hilos de ejecución mediante directivas, facilitando la paralelización de fragmentos de código sin necesidad de construir manualmente toda la lógica de gestión de hilos. En ParaLab, OpenMP se utiliza como herramienta base para introducir al estudiante en el paralelismo dentro de una sola máquina (OpenMP Architecture Review Board, 2024).
- Hilos de ejecución: son unidades de ejecución dentro de un proceso que pueden correr de manera concurrente o paralela. En el modelo de memoria compartida, varios hilos pueden trabajar sobre el mismo programa y acceder a variables comunes, lo que permite distribuir tareas, pero también requiere controlar adecuadamente la sincronización.
- Sincronización: es el conjunto de mecanismos que permiten coordinar la ejecución de varios hilos o procesos para evitar resultados incorrectos. La sincronización es necesaria cuando varias tareas acceden a recursos compartidos, como variables, archivos o estructuras de datos. En el aprendizaje de concurrencia, este concepto es clave para comprender problemas como condiciones de carrera, bloqueos y errores de consistencia.
- Memoria distribuida: es un modelo de programación paralela en el que varios procesos se ejecutan en espacios de memoria independientes. A diferencia del modelo de memoria compartida, los procesos no acceden directamente a los datos de los demás, por lo que deben comunicarse mediante mensajes. Este modelo es común en sistemas distribuidos y clústeres de cómputo.
- MPI: Message Passing Interface es un estándar utilizado para la programación paralela en sistemas de memoria distribuida. Permite que varios procesos se comuniquen entre sí mediante el envío y recepción de mensajes. En ParaLab, MPI se utiliza para que los estudiantes experimenten con ejecución distribuida sobre contenedores que simulan nodos de cómputo (Message Passing Interface Forum, 2025).
- Procesos: son instancias de ejecución de un programa que cuentan con su propio espacio de memoria. En el modelo de memoria distribuida, cada proceso ejecuta una parte del trabajo y se comunica con otros procesos mediante mensajes. Este concepto permite comprender cómo se distribuye una tarea en un entorno compuesto por varios nodos o unidades de ejecución independientes.
- Comunicación entre procesos: es el mecanismo mediante el cual diferentes procesos intercambian información durante la ejecución de un programa distribuido. En MPI, esta comunicación ocurre a través de operaciones de envío y recepción de mensajes. Comprender este concepto permite analizar cómo se coordinan tareas en entornos donde no existe memoria compartida.
- Nodos de cómputo: son unidades encargadas de ejecutar tareas dentro de un entorno distribuido. En un clúster físico, un nodo puede corresponder a una máquina real. En ParaLab, los nodos son simulados mediante contenedores, lo que permite representar de forma controlada un entorno de ejecución distribuida sin requerir varias máquinas físicas.
- Contenedores: son entornos aislados que permiten empaquetar una aplicación junto con sus dependencias, bibliotecas y configuraciones necesarias para su ejecución. Su

uso facilita la portabilidad y reproducibilidad del entorno, ya que permite ejecutar una misma configuración en diferentes equipos con menor dependencia de instalaciones manuales.

- Docker: es una plataforma que permite crear, ejecutar y administrar contenedores. En el contexto de ParaLab, Docker puede utilizarse como base tecnológica para simular nodos de cómputo y construir un entorno distribuido donde se ejecuten programas MPI. Su papel dentro del proyecto es servir como medio de experimentación, no como objeto principal de aprendizaje.
- Virtualización ligera: es una forma de aislamiento de entornos que no requiere virtualizar completamente un sistema operativo. Los contenedores son considerados una forma de virtualización ligera porque comparten el sistema operativo anfitrión, pero aíslan aplicaciones y dependencias. Esto los hace más livianos que las máquinas virtuales tradicionales y adecuados para crear entornos educativos reproducibles.
- Simulación de nodos de cómputo: consiste en representar varios nodos mediante contenedores dentro de una misma máquina física. Esta simulación no busca reemplazar un clúster real de alto rendimiento, sino ofrecer un entorno controlado que permita experimentar con conceptos de memoria distribuida, comunicación entre procesos y ejecución paralela.
- Entorno controlado de experimentación: es un espacio diseñado para que los estudiantes puedan ejecutar programas, modificar configuraciones, observar resultados y repetir pruebas sin depender de infraestructura especializada. En ParaLab, este entorno busca facilitar el aprendizaje progresivo del paralelismo y la concurrencia.
- Métricas de rendimiento: son indicadores que permiten evaluar el comportamiento de una solución computacional. En el contexto de programación paralela, estas métricas ayudan a determinar si una implementación paralela mejora realmente el desempeño frente a una versión secuencial.
- Tiempo de ejecución: es la duración que tarda un programa en completar su procesamiento. Es una de las métricas básicas para comparar una versión secuencial con una versión paralela. En ParaLab, esta métrica permite que el estudiante observe cómo cambia el rendimiento al modificar la configuración de ejecución.
- Aceleración o speedup: es una métrica que compara el tiempo de ejecución de una solución secuencial con el tiempo de ejecución de una solución paralela. Permite analizar cuánto más rápida es una versión paralela respecto a la versión original. Su interpretación es fundamental para comprender si el paralelismo aporta una mejora real.
- Eficiencia: es una métrica que relaciona la aceleración obtenida con la cantidad de recursos utilizados. Permite analizar qué tan bien se aprovechan los hilos, procesos o nodos disponibles. Una solución puede ser más rápida que la secuencial, pero no necesariamente eficiente si utiliza demasiados recursos para obtener una mejora pequeña.
- Escalabilidad: es la capacidad de un programa o sistema para mantener o mejorar su rendimiento cuando se incrementan los recursos disponibles, como hilos, procesos o nodos. En ParaLab, este concepto permite que el estudiante analice cómo cambia el comportamiento de un programa al aumentar la cantidad de recursos de ejecución.

- Ley de Amdahl: es un principio utilizado para analizar el límite teórico de mejora que puede obtenerse al paralelizar un programa. Indica que la aceleración máxima está limitada por la parte del programa que necesariamente debe ejecutarse de manera secuencial. Este concepto ayuda a comprender por qué no todos los programas mejoran proporcionalmente al aumentar el número de recursos.
- Interfaz gráfica interactiva: es el componente visual que permite al usuario interactuar con la plataforma sin depender exclusivamente de comandos o archivos de configuración. En ParaLab, la interfaz gráfica busca facilitar la configuración del entorno, la ejecución de programas y la visualización de métricas de rendimiento.
- Módulo educativo: es la sección de la plataforma orientada a apoyar el aprendizaje del estudiante mediante contenidos, explicaciones, talleres, ejercicios guiados y recursos audiovisuales. En ParaLab, este módulo complementa la ejecución de programas con material formativo que permite comprender los conceptos trabajados.
- Talleres prácticos guiados: son actividades estructuradas paso a paso que permiten al estudiante aplicar conceptos de paralelismo y concurrencia mediante ejercicios concretos. Estos talleres ayudan a conectar la teoría con la práctica, orientando al estudiante en la ejecución, observación e interpretación de resultados.
- Recursos audiovisuales: son materiales educativos en formato de video que apoyan la explicación de conceptos, herramientas y procedimientos de uso de la plataforma. En ParaLab, estos recursos pueden utilizarse para introducir temas como memoria compartida, memoria distribuida, sincronización, comunicación entre procesos y métricas de rendimiento.
- Aprendizaje práctico: es una forma de aprendizaje basada en la experimentación directa. En el contexto de ParaLab, implica que el estudiante no solo lea o escuche explicaciones sobre paralelismo y concurrencia, sino que ejecute programas, observe resultados, modifique configuraciones y analice métricas.
- Pertinencia educativa: hace referencia al grado en que una herramienta responde a las necesidades de aprendizaje del contexto para el cual fue diseñada. En este proyecto, la pertinencia educativa de ParaLab se relaciona con su capacidad para apoyar la comprensión práctica de conceptos de paralelismo y concurrencia en estudiantes de ingeniería y áreas afines.
- Usabilidad: es el grado en que un sistema puede ser utilizado por usuarios específicos para alcanzar objetivos concretos con eficacia, eficiencia y satisfacción. En ParaLab, la usabilidad es importante porque la plataforma debe facilitar el aprendizaje y no convertirse en una barrera técnica adicional para el estudiante.
- Pruebas funcionales: son pruebas que permiten verificar que las funciones principales del sistema operen de acuerdo con los requisitos definidos. En este proyecto, estas pruebas permiten comprobar la ejecución de programas OpenMP, la ejecución de programas MPI sobre contenedores, la visualización de métricas y el funcionamiento general de la interfaz.
- Validación educativa: es el proceso mediante el cual se analiza si la plataforma cumple con su propósito formativo. Puede incluir actividades con usuarios, casos de prueba, revisión por docentes o retroalimentación de estudiantes. Su objetivo es determinar si ParaLab resulta útil y comprensible como apoyo al aprendizaje práctico del paralelismo y la concurrencia.

5.2 Análisis de alternativas de solución

En esta sección se presentan y analizan diferentes alternativas relacionadas con la enseñanza práctica de programación paralela, computación distribuida y uso de contenedores en contextos educativos. Estas alternativas permiten ubicar la propuesta ParaLab frente a enfoques existentes, como clústeres físicos de bajo costo, entornos virtualizados, plataformas basadas en contenedores o soluciones orientadas a la ejecución de aplicaciones paralelas.

El análisis de alternativas permite identificar ventajas, limitaciones y oportunidades de mejora en relación con la solución propuesta. En particular, se consideran criterios como accesibilidad, complejidad técnica, requerimientos de infraestructura, facilidad de uso, enfoque educativo, soporte para memoria compartida y distribuida, visualización de métricas y posibilidad de reproducir el entorno en diferentes equipos.

5.2.1 Alternativas de solución e impacto

Alternativa 1: Clústeres físicos de bajo costo para enseñanza de HPC

Una alternativa para facilitar la enseñanza práctica de computación paralela, presentada por Demay et al. (2024), consiste en implementar clústeres físicos de bajo costo dentro de instituciones educativas. Este tipo de solución permite que los estudiantes interactúen con una infraestructura real, compuesta por varios nodos de cómputo, redes de interconexión y herramientas de administración de recursos.

Desde el punto de vista técnico, su principal ventaja es que ofrece una experiencia cercana a un entorno real de computación de alto rendimiento. Los estudiantes pueden observar la distribución de tareas, la comunicación entre nodos, la asignación de recursos y los retos asociados al uso de infraestructura física. Sin embargo, este enfoque requiere adquirir equipos, configurar redes, mantener hardware, instalar software especializado y contar con soporte técnico permanente.

Desde el punto de vista educativo, los clústeres físicos pueden ser valiosos para cursos avanzados o instituciones que ya cuentan con infraestructura disponible. No obstante, para contextos introductorios o programas con recursos limitados, pueden representar una barrera de entrada alta. En comparación, ParaLab busca ofrecer una experiencia práctica más accesible, centrada en el aprendizaje progresivo de OpenMP, MPI y métricas de rendimiento, sin depender de infraestructura física especializada.

Alternativa 2: Entornos virtualizados para enseñanza de computación paralela y distribuida

Otra alternativa consiste en utilizar máquinas virtuales o plataformas de virtualización para crear entornos de aprendizaje relacionados con computación paralela, sistemas distribuidos o infraestructura tipo clúster. Estas soluciones permiten aislar configuraciones, administrar recursos y ofrecer a los estudiantes ambientes controlados para experimentar con distintas tecnologías.

Desde el punto de vista técnico, los entornos virtualizados ofrecen mayor aislamiento que los contenedores y pueden simular sistemas completos con sus propios sistemas operativos. Sin embargo, suelen requerir más recursos computacionales, mayor tiempo de configuración y una administración más compleja. Además, cuando se busca ejecutar varias instancias al mismo tiempo, el consumo de CPU, memoria y almacenamiento puede aumentar considerablemente.

Desde el punto de vista educativo, la virtualización puede ser útil para enseñar administración de sistemas, redes o infraestructura. Sin embargo, para el aprendizaje práctico del paralelismo y la concurrencia, una solución basada en contenedores puede resultar más ligera y fácil de desplegar. ParaLab aprovecha esta ventaja al utilizar contenedores como medio para simular nodos de cómputo y permitir la ejecución de programas MPI sin requerir máquinas virtuales completas.

Alternativa 3: Entornos basados en contenedores para aprendizaje de programación paralela

Una tercera alternativa corresponde a los entornos educativos basados en contenedores, como la propuesta de Arisal y Suhartanto (2024), diseñados para permitir la ejecución de programas paralelos desde un equipo personal o institucional. Este enfoque resulta especialmente relevante para el proyecto, ya que los contenedores permiten encapsular dependencias, bibliotecas y configuraciones, facilitando la portabilidad y reproducibilidad del entorno.

Desde el punto de vista técnico, esta alternativa reduce los problemas asociados con la instalación manual de herramientas como compiladores, bibliotecas de programación paralela o configuraciones de red. Además, permite simular varios nodos mediante contenedores conectados entre sí, lo que resulta útil para experimentar con memoria distribuida y comunicación entre procesos.

Desde el punto de vista educativo, los entornos basados en contenedores permiten que los estudiantes practiquen con programación paralela sin depender de un clúster físico. Sin embargo, algunas soluciones existentes se centran principalmente en la ejecución de código y no necesariamente integran una ruta pedagógica guiada, visualización de métricas o recursos educativos complementarios. En ese punto, ParaLab se diferencia al proponer una plataforma que articula ejecución, medición e interpretación de resultados dentro de una experiencia educativa progresiva.

Alternativa 4: Plataformas basadas en notebooks para programación paralela

Otra alternativa utilizada en contextos educativos es el uso de notebooks interactivos, como Jupyter Notebook, para enseñar programación paralela (Arisal & Suhartanto, 2024). Este enfoque permite combinar explicación, código ejecutable y resultados en un mismo entorno, lo que facilita la experimentación inmediata y el aprendizaje autónomo.

Desde el punto de vista educativo, los notebooks son valiosos porque permiten al estudiante seguir ejemplos paso a paso, modificar código y observar resultados rápidamente. Además, son útiles para presentar explicaciones teóricas junto con ejercicios prácticos. No obstante, su alcance puede ser limitado cuando se requiere representar una arquitectura distribuida más

explícita, configurar nodos simulados o visualizar métricas de rendimiento de manera integrada en una plataforma diseñada específicamente para el aprendizaje.

ParaLab puede tomar como referencia las ventajas de los notebooks en cuanto a aprendizaje guiado e interacción, pero plantea una solución más orientada a integrar distintos componentes en una misma plataforma: ejecución OpenMP, ejecución MPI sobre contenedores, configuración mediante interfaz gráfica, visualización de métricas y módulo educativo.

Alternativa 5: Uso directo de clústeres institucionales o de nube

Otra alternativa consiste en que los estudiantes aprendan paralelismo y computación distribuida directamente sobre clústeres institucionales o servicios de nube. Este enfoque tiene la ventaja de acercar al estudiante a entornos más reales y escalables, donde puede interactuar con recursos de mayor capacidad y herramientas utilizadas en contextos profesionales o de investigación.

Sin embargo, esta alternativa presenta barreras importantes en etapas iniciales de aprendizaje. El acceso puede depender de permisos institucionales, disponibilidad de recursos, costos de uso, configuración de cuentas, políticas de seguridad y soporte técnico. Además, para estudiantes que apenas se están acercando a la programación paralela, la complejidad del entorno puede dificultar la comprensión de los conceptos fundamentales.

En comparación, ParaLab no busca reemplazar estos entornos avanzados, sino funcionar como un paso previo de aprendizaje. Su propósito es permitir que el estudiante comprenda y practique conceptos básicos e intermedios de paralelismo y concurrencia en un ambiente controlado, antes de enfrentarse a infraestructuras más complejas.

5.2.2 Comparación de alternativas

A partir de las alternativas analizadas, se evidencia que existen distintos enfoques para apoyar el aprendizaje práctico del paralelismo, la concurrencia y la programación distribuida. Los clústeres físicos ofrecen una experiencia cercana a entornos reales, pero requieren infraestructura, mantenimiento y soporte especializado. Los entornos virtualizados permiten aislar configuraciones completas, pero suelen ser más pesados y complejos de administrar. Las soluciones basadas en contenedores ofrecen portabilidad y reproducibilidad, aunque algunas se limitan a la ejecución de código sin integrar una ruta educativa completa. Los notebooks facilitan la experimentación guiada, pero no siempre representan de forma clara una arquitectura distribuida. Finalmente, los clústeres institucionales o de nube ofrecen mayor realismo, pero pueden ser difíciles de usar como primer acercamiento.

Frente a estas alternativas, ParaLab se diferencia por integrar en una misma plataforma un enfoque progresivo de aprendizaje, ejecución práctica con OpenMP y MPI, uso de contenedores para simular nodos de cómputo, visualización de métricas de rendimiento y recursos educativos de apoyo. Esta integración permite que el estudiante no solo ejecute programas paralelos, sino que también observe su comportamiento, compare configuraciones, interprete resultados y relacione la teoría con la práctica.

Alternativa	Infraestructura requerida	Complejidad técnica	Soporte OpenMP	Soporte MPI	Uso de contenedores	Visualización de métricas	Módulo educativo	Adecuación para aprendizaje inicial
Clúster físico de bajo costo	Media/alta	Alta	Posible	Sí	No necesariamente	Depende de configuración	Depende del curso	Media
Entorno virtualizado	Media	Media/alta	Posible	Posible	No necesariamente	Depende de implementación	Limitado	Media
Entorno basado en contenedores	Baja/media	Media	Posible	Sí	Sí	Limitada o variable	Limitado	Alta
Notebooks interactivos	Baja/media	Baja/media	Posible	Limitado o configurable	No necesariamente	Básica	Alta	Alta
Clúster institucional o nube	Alta	Alta	Sí	Sí	Variable	Depende de herramientas	No necesariamente	Baja/media
ParaLab	Baja/media	Media	Sí	Sí	Sí	Sí	Sí	Alta

Tabla 2. Comparación de alternativas. Elaboración propia.

La Tabla 2 presenta una comparación entre las alternativas analizadas y la propuesta ParaLab. En ella se pueden considerar criterios como infraestructura requerida, complejidad técnica, facilidad de despliegue, soporte para OpenMP, soporte para MPI, uso de contenedores, visualización de métricas, integración de recursos educativos y pertinencia para contextos universitarios. Esta comparación permite evidenciar que ParaLab busca un equilibrio entre accesibilidad técnica, valor educativo y experimentación práctica, manteniendo como eje central el aprendizaje del paralelismo y la concurrencia.

6 Referencias

- Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. In AFIPS Conference Proceedings, 30, 483–485.
<https://doi.org/10.1145/1465482.1465560>
- Arisal, A., & Suhartanto, H. (2024). HPC-in-Containers: A containerized parallel environment for parallel programming learning using Docker. In 2024 International Conference on Information Technology Systems and Innovation (ICITSI) (pp. 101–105). IEEE. <https://doi.org/10.1109/ICITSI65188.2024.10929375>
- Barrios-Hernandez, C. J., Rivas, R., Besseron, X., Diaz, G., Georgiou, Y., & Velho, P. (2024). High-performance computing training by skills for advanced computing ecosystems. *Revista UIS Ingenierías*, 23(1), 175–188.
<https://doi.org/10.18273/revuin.v23n1-2024014>
- Chung, M. T., Nguyen, Q.-H., Nguyen, M.-T., & Thoai, N. (2016). Using Docker in high performance computing applications. In 2016 IEEE Sixth International Conference on Communications and Electronics (ICCE) (pp. 52–57). IEEE.
<https://doi.org/10.1109/CCE.2016.7562612>
- Demay, T. A. O., Lima, L. S. C., Fornaciali, M. S., Batista, A. F. M., & Ferrão, R. C. (2024). Cost-effective clusters in education: A model for practical HPC learning. In 2024 IEEE Frontiers in Education Conference (FIE). IEEE.
<https://doi.org/10.1109/FIE61694.2024.10892965>
- Greneche, N., & Cerin, C. (2022). Autoscaling of containerized HPC clusters in the cloud. In 2022 IEEE/ACM International Workshop on Interoperability of Supercomputing and Cloud Technologies (SuperCompCloud) (pp. 1–7). IEEE.
<https://doi.org/10.1109/SuperCompCloud56703.2022.00006>
- Hebert, J., Hratish, R., Gomes, R., Kunkel, W., Marshall, D., Ghosh, A., Doss, I., Ma, Y., Stedman, D., Stinson, B., Varghese, A., Mohr, M., Rozario, P., & Bhattacharyya, S. (2024). High-performance computing in undergraduate education at primarily undergraduate institutions in Wisconsin: Progress, challenges, and opportunities. *Education and Information Technologies*, 29, 18451–18475.
<https://doi.org/10.1007/s10639-024-12582-6>
- IEEE Computer Society. (2009). IEEE standard for information technology—Systems design—Software design descriptions (IEEE Std 1016-2009). IEEE.
- IEEE Standards Association. (2020). IEEE standard for learning object metadata (IEEE Std 1484.12.1-2020). IEEE.
- International Organization for Standardization. (2016). Systems and software engineering—Systems and software Quality Requirements and Evaluation (SQuaRE)—Measurement of system and software product quality (ISO/IEC 25023:2016). ISO.
- International Organization for Standardization. (2018). Ergonomics of human-system interaction—Part 11: Usability: Definitions and concepts (ISO 9241-11:2018). ISO.

- International Organization for Standardization, International Electrotechnical Commission, & Institute of Electrical and Electronics Engineers. (2022). Software and systems engineering—Software testing—Part 1: General concepts (ISO/IEC/IEEE 29119-1:2022). ISO.
- Kurtzer, G. M., Sochat, V., & Bauer, M. W. (2017). Singularity: Scientific containers for mobility of compute. *PLOS ONE*, 12(5), e0177459. <https://doi.org/10.1371/journal.pone.0177459>
- Message Passing Interface Forum. (2025). MPI: A message-passing interface standard, version 5.0. <https://www.mpi-forum.org/docs/mpi-5.0/mpi50-report.pdf>
- Ngo, L. B., & Kilgannon, J. (2020). Virtual cluster for HPC education. *Journal of Computing Sciences in Colleges*, 36(3), 20–30. <https://doi.org/10.5555/3447080.3447084>
- Object Management Group. (2014). Business Process Model and Notation (BPMN), version 2.0.2. <https://www.omg.org/spec/BPMN/2.0.2>
- Object Management Group. (2017). Unified Modeling Language (UML), version 2.5.1. <https://www.omg.org/spec/UML/2.5.1>
- Open Container Initiative. (2025). Open Container Initiative runtime specification, version 1.3.0. <https://opencontainers.org/posts/blog/2025-11-04-oci-runtime-spec-v1-3/>
- OpenMP Architecture Review Board. (2024). OpenMP application programming interface, version 6.0. <https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-6-0.pdf>
- Sheka, A., Bersenev, A., & Samun, V. (2019). The problem of reproducible results on the HPC cluster. In 2019 International Multi-Conference on Engineering, Computer and Information Sciences (SIBIRCON) (pp. 833–837). IEEE. <https://doi.org/10.1109/SIBIRCON48586.2019.8957856>
- Zhou, N., Zhou, H., & Hoppe, D. (2023). Containerization for high performance computing systems: Survey and prospects. *IEEE Transactions on Software Engineering*, 49(4), 2722–2740. <https://doi.org/10.1109/TSE.2022.3229221>